

Question 1 — IoT Sensor Dashboard

Topics: Data types · Operators · Conditionals · Loops · Jump Statements · 1D & 2D Arrays · Command-Line Arguments

Problem 1 — Sensor Reading Classifier

You have joined a team building firmware for an industrial factory monitoring system. The factory floor has temperature sensors installed near heavy machinery. Each sensor sends a single `double` reading (in °C) to the control unit every few seconds.

Your task is to write the **classification module** — the piece of code that reads one sensor value, maps it to a status, and prints what action the control unit should take.

Range (°C)	Status Code	Status Label	Action
< 0	-1	SENSOR_ERROR	Sensor fault — check wiring
0 – 29	0	NORMAL	No action required
30 – 44	1	WARNING	Alert sent to supervisor
45 – 59	2	CRITICAL	Cooling system triggered
≥ 60	3	SHUTDOWN	Emergency shutdown initiated

Requirements:

- Store the reading as `double`, derive a status code as `int` using `if-else`
- Use `switch` on the status code to print the action
- Use the **ternary operator** to print `Above Average` or `Below Average` relative to 25°C (normal operating temperature)
- Print the temperature in Fahrenheit as well: $F = (C \times 9 / 5) + 32$

Expected Output (input: 47.3):

```
Temperature : 47.3°C / 117.14°F
Status      : CRITICAL
Action      : Cooling system triggered
Reading     : Above Average
```

Problem 2 — Sensor Log Buffer

The factory sensors now send one reading per second. Your senior engineer tells you:

"We can't store everything — just keep the last N readings in a buffer and run basic analysis on them."

You are asked to write the **buffer analysis module**. It stores readings in a fixed-size array and generates a status report.

Requirements:

1. Accept **N** from the user ($1 \leq N \leq 100$), then read N temperature values into an array
2. Print all valid readings — skip values below 0 (sensor error) using **continue**
3. Scan for the **first reading at or above 45°C** — print its index and stop scanning using **break**
4. Compute min, max, and average in **one single loop pass**
5. Count readings per category: Normal / Warning / Critical / Shutdown

Expected Output (sample data):

```
Readings entered : 8
Valid readings   : 22.1  31.5  46.0  28.0  50.2  10.0  38.0
Skipped (errors) : 1

First CRITICAL   : Index 3  →  46.0°C

Min : 10.0°C     Max : 50.2°C     Avg : 32.26°C

Normal:2  Warning:3  Critical:2  Shutdown:0
```

Problem 3 — Building Sensor Grid

The system has been scaled to a full smart building. Three floors, three rooms per floor — a 3×3 grid of sensors. The building manager wants a **floor-by-floor status report** on his dashboard every morning.

You are assigned to write the grid reading and reporting module.

Requirements:

1. Read temperatures for all 9 rooms into a 2D array (rows = floors, columns = rooms)
2. Display the readings in a formatted table
3. Find and report the hottest room (floor and room number)
4. Find and report the floor with the highest average temperature
5. Count total rooms at or above the WARNING threshold (30°C)

Expected Output (sample):

	Room1	Room2	Room3
Floor 1	: 24.0	31.5	28.0
Floor 2	: 45.0	22.0	30.0
Floor 3	: 19.0	27.5	50.2

Hottest Room : Floor 3, Room 3 → 50.2°C

Hottest Floor : Floor 2 (avg 32.33°C)

Rooms at WARNING or above : 4

Problem 4 — Startup Configuration via CLI

The monitoring program now runs as a **background daemon on an embedded Linux device**. The operations team does not want to recompile the code every time thresholds change — they want to pass thresholds at startup via the command line, the same way they configure `nginx` or any other service.

You are asked to write the configurable version of the monitor.

Program name: `sensor_monitor`

Usage:

```
./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>
```

Requirements:

- If arguments are missing, print a usage line and exit with code `1`
- Validate: `warn < critical, 1 ≤ num_readings ≤ 500` — print a specific error and exit on failure
- If valid, simulate `num_readings` temperature values using `rand() % 70`, classify each using the provided thresholds, and print a category summary

Expected Output:

```
$ ./sensor_monitor 30 45 10
Config : Warn=30°C Critical=45°C Readings=10
Results : Normal:4 Warning:3 Critical:2 Shutdown:1

$ ./sensor_monitor 30
Usage : ./sensor_monitor <warn_threshold> <critical_threshold> <num_readings>
Error : Missing arguments.
```

Question 2 — Embedded Memory Manager & Signal Processor

Topics: Functions (all 4 forms) · Call by Value · Call by Reference · Call by Pointer · Reference Variables · Inline Functions · `<cmath>` · Pointers · Pointer Arithmetic · `const` + Pointer Combinations · Pointer to Pointer (`**`)

Problem 1 — The Bug in Sensor Recalibration

Your team lead assigns you a code review task. A junior developer submitted a function `resetSensorPair()` — it is supposed to swap two sensor readings before recalibration runs. During testing, the sensors are not swapping. The team lead says:

"Figure out why it doesn't work and give me two fixed versions — one using references, one using pointers. Then write a comment explaining the root cause. We'll use it in the next team knowledge-sharing session."

Write all three versions:

```
void resetSensorPairV1(int reading1, int reading2);    // Original bug – Call by
Value
void resetSensorPairV2(int& reading1, int& reading2);  // Fix 1 – Call by
Reference
void resetSensorPairV3(int* reading1, int* reading2);  // Fix 2 – Call by Pointer
```

Call all three from `main()`. Print values before and after each call. Write a comment block (minimum 4 lines) inside `main()` explaining why V1 fails.

Expected Output:

```
--- V1: Call by Value ---
Before : A=55 B=12
After  : A=55 B=12      ← values unchanged

--- V2: Call by Reference ---
Before : A=55 B=12
After  : A=12 B=55      ← values swapped

--- V3: Call by Pointer ---
Before : A=12 B=55
After  : A=55 B=12      ← values swapped back
```

Problem 2 — Signal Processing Pipeline

You are building a pre-processing module for an audio analysis system. Raw audio data arrives as an array of `double` samples. Before any AI model can process it, the signal must go through a fixed pipeline of operations.

Your senior writes the function signatures and says:

"No array indexing inside these functions. Use pointer arithmetic — that's how the DSP library we're interfacing with operates."

```
double computeRMS(double* signal, int n);
// Returns sqrt( sum of (each element squared) / n )

void normalise(double* signal, int n);
// Divides every element by the max absolute value in the array (in-place)

int countZeroCrossings(double* signal, int n);
// Returns count of positions where adjacent elements have opposite signs

void applyGain(double* signal, int n, double gainFactor);
// Multiplies every element by gainFactor (in-place)
```

Rule: No `arr[i]` inside any of these functions. Use `*ptr`, `*(ptr + i)`, or `ptr++` only.

Test signal: {0.5, -1.2, 0.8, -0.3, 1.0, -0.9, 0.1}

Print the array before and after calling `normalise()` and `applyGain()`. Print the value returned by `computeRMS()` and `countZeroCrossings()`.

Problem 3 — Hardware Register Access

You are writing low-level firmware that communicates with a sensor chip. The chip has three types of registers mapped to memory addresses:

- **Status register** — the firmware must only *read* this (the chip writes to it)
- **Control register** — the firmware writes to it, and it must always point to the same address
- **ROM config** — fixed value burned at factory, never changes

If a developer accidentally writes to the status register, the chip enters an undefined state. Your job is to enforce these access rules using `const` pointer types.

```
int statusReg = 0b10110001; // Read-only from firmware side
int controlReg = 0b00000000; // Firmware writes here
int dataReg = 0b11001010; // For reassignment demo
```

Requirements:

- Declare `regPtr1` as `const int*` pointing to `statusReg` — print its value; attempt a write and a repoint, comment both out with error explanation
- Declare `regPtr2` as `int* const` pointing to `controlReg` — write a new value through it; attempt a repoint, comment it out with explanation
- Declare `regPtr3` as `const int* const` pointing to `statusReg` — print its value; attempt both write and repoint, comment both out with explanations

Problem 4 — Calibration Packet Parser

Your embedded device receives raw integer data packets over a serial bus. Before the main application can use the data, a parser must scan each packet and locate the minimum and maximum values for sensor calibration. These two values must be handed back to the caller.

The protocol team says: *"Don't copy the values — give back pointers into the original packet buffer. We need to know their positions too."*

```
bool parsePacket(const int* rawData, int size,
                 int** outMin, int** outMax);
// If size <= 0 : return false, leave output pointers unchanged
// Otherwise    : set *outMin to point at the minimum element in rawData
//               set *outMax to point at the maximum element in rawData
//               return true
```

main() — use exactly as given:

```
int packet[] = {45, 12, 67, 8, 55, 31};
int* minPtr = nullptr;
int* maxPtr = nullptr;

if (parsePacket(packet, 6, &minPtr, &maxPtr)) {
    cout << "Calibration Min : " << *minPtr << endl;
    cout << "Calibration Max : " << *maxPtr << endl;
}
```

Expected Output:

```
Calibration Min : 8
Calibration Max : 67
```

Problem 5 — Drone Navigation Utilities

You are part of a team building the flight controller for an autonomous delivery drone. The navigation module needs a small set of math helper functions used hundreds of times per second. Your lead engineer says:

*"Write these as **inline** functions. We don't want function call overhead in the navigation loop — the compiler should inline the code directly."*

```
inline double distanceBetween(double x1, double y1, double x2, double y2);
// sqrt( pow(x2-x1, 2) + pow(y2-y1, 2) )

inline double toRadians(double degrees);
// degrees * (M_PI / 180.0)
```

```
inline double clamp(double value, double minVal, double maxVal);  
// Restrict value to [minVal, maxVal]  
  
inline bool isInSafeZone(double x, double y, double cx, double cy, double radius);  
// true if point (x,y) is within the circle centred at (cx,cy) with given radius
```

In `main()`, set home position as `(0.0, 0.0)` and safe-zone radius as `50.0` units. Test with 3 waypoints of your choice. For each waypoint print its distance from home and whether it is inside the safe zone.

Question 3 — HR Payroll Engine

Topics: Classes · Objects · Access Specifiers · Data Members · Member Functions · Getters (`const`) · Setters (with validation) · Encapsulation · Abstraction · Stack vs Heap Objects (`.` vs `->`) · `static` Members · `delete`

The Scenario

Your company has won a contract to build an HR module for a mid-sized manufacturing firm. The existing system stores employee data directly in global variables — anyone in the codebase can accidentally set a salary to a negative number or assign an employee to a non-existent department, causing payroll errors at month end.

Your task is to redesign this as a proper C++ class that **enforces data rules at the point of entry**, so invalid data never enters the system in the first place.

Class: `Employee`

Private data members:

Member	Type	Rule
<code>empId</code>	<code>int</code>	Auto-assigned starting from 1001, increments per object created
<code>name</code>	<code>string</code>	Cannot be empty
<code>department</code>	<code>string</code>	Must be one of: Engineering, HR, Finance, Operations
<code>grade</code>	<code>char</code>	Only 'A', 'B', 'C', 'D' accepted
<code>basicSalary</code>	<code>double</code>	Must be > 10,000 and < 5,00,000
<code>isActive</code>	<code>bool</code>	Default <code>true</code> ; only <code>deactivate()</code> can set it to <code>false</code>
<code>employeeCount</code>	<code>static int</code>	Shared across all objects — counts total employees created

Public setters — each validates input and prints an error if the value is rejected:

```
void setName(const string& n);
void setDepartment(const string& dept);
void setGrade(char g);
void setBasicSalary(double salary);
void deactivate();
```

Public getters — all marked `const`:

```
int    getEmpId()      const;
string getName()       const;
string getDepartment() const;
char   getGrade()      const;
double getBasicSalary() const;
bool   getIsActive()   const;
```

Business logic — all marked `const`:

```
double computeAllowances() const;
// Grade A → 40% of basicSalary (Senior / Manager)
// Grade B → 30% of basicSalary (Mid-level)
// Grade C → 20% of basicSalary (Junior)
// Grade D → 10% of basicSalary (Trainee)

double computeGrossSalary() const;
// basicSalary + computeAllowances()

double computeTax() const;
// Gross <= 50,000           → 0%
// 50,001 to 1,00,000       → 10% on amount above 50,000
// > 1,00,000               → 5,000 + 20% on amount above 1,00,000

double computeNetSalary() const;
// computeGrossSalary() - computeTax()

void printPayslip() const;

static int getEmployeeCount();
```

Input:

```
void acceptDetails();
// Reads all fields from the user – must call the corresponding setter for each
// so that validation runs before any value is stored
```


main() — Required Structure

```
int main() {  
  
    // Create objects – one on stack, two on heap  
    Employee e1;  
    Employee* e2 = new Employee();  
    Employee* e3 = new Employee();  
  
    e1.acceptDetails();  
    e2->acceptDetails();  
    e3->acceptDetails();  
  
    // Try uncommenting – observe the compiler error, then explain in a comment  
    why  
    // e1.empId = 999;  
    // e1.basicSalary = -1000;  
  
    e1.printPayslip();  
    e2->printPayslip();  
    e3->printPayslip();  
  
    // Simulate a resignation  
    e3->deactivate();  
    if (!e3->getIsActive())  
        cout << e3->getName() << " is no longer active. Payroll skipped." << endl;  
  
    cout << "Total Employees : " << Employee::getEmployeeCount() << endl;  
  
    delete e2;  
    delete e3;  
  
    return 0;  
}
```

Validation — Required in Output

```
Enter grade: X  
ERROR: Invalid grade 'X'. Accepted values: A, B, C, D.  
  
Enter basic salary: -5000  
ERROR: Salary must be between Rs.10,000 and Rs.5,00,000. Value rejected.  
  
Enter department: Marketing  
ERROR: 'Marketing' is not a registered department.
```

Sample Payslip Output

```
=====
EMPLOYEE PAYSALIP – AUG 2026
=====
Emp ID      : 1001
Name        : Priya Sharma
Department  : Engineering
Grade       : B
Status      : Active
-----
Basic Salary    : Rs. 75,000.00
Allowances (30%) : Rs. 22,500.00
Gross Salary    : Rs. 97,500.00
-----
Tax Deduction   : Rs. 4,750.00
Net Salary      : Rs. 92,750.00
=====
```

Bonus — Struct Padding (Optional, 5 marks)

During a code review, a colleague asks why two structs with the same members but different order have different sizes. You need to explain it.

```
struct Layout1 { char c1; int i; char c2; };
struct Layout2 { int i; char c1; char c2; };
```

Print `sizeof(Layout1)` and `sizeof(Layout2)`. Inside a comment block, explain:

1. Why the sizes differ
2. What padding is and why the compiler adds it
3. Why member order matters when defining network packet headers or hardware register maps